

Page · the loop: outline, draft, probe, evidence, revise, compile, check

1 · OVERVIEW: WHAT THE LOOP IS, AND HOW TO READ THE REST

The claim this page makes: a phase is defined by the ONE question it answers, not by the edit it performs, and a Page moves forward by satisfying a phase's exit rather than by being worked on. The problem: one Page is worked by many hands over many days, and the same visible edit can be two different acts. Adding a paragraph under an existing Aim and adding a paragraph that creates one look identical in a diff. So the loop names acts by their REASON, never by their diff. Each phase is the question it answers: OUTLINE asks what shape, PROBE asks what is unknown, EVIDENCE asks what backs it, DRAFT asks how it is said, REVISE asks how it gets better, and CHECK asks where it goes next. Progress is satisfying a phase's exit, a tick, a landed card, a verdict; hours of editing move nothing without the exit. The Opening says where this page sits among its siblings. This division says what the LOOP is. Neither repeats the other.

The loop is not a line: QPw00¹-Display4 (Figure 1) draws it:
phases ①, ② and ③ repeat as one converging PREPARE loop,
and only a person's approved: tick on the plan releases the page
into ④, ⑤ and ⑦.

1 · haipipe

Display: QPw00-Display4-the-loop-at-a-glance · kind=figure · label=fig:qpw00-loop-at-a-glance

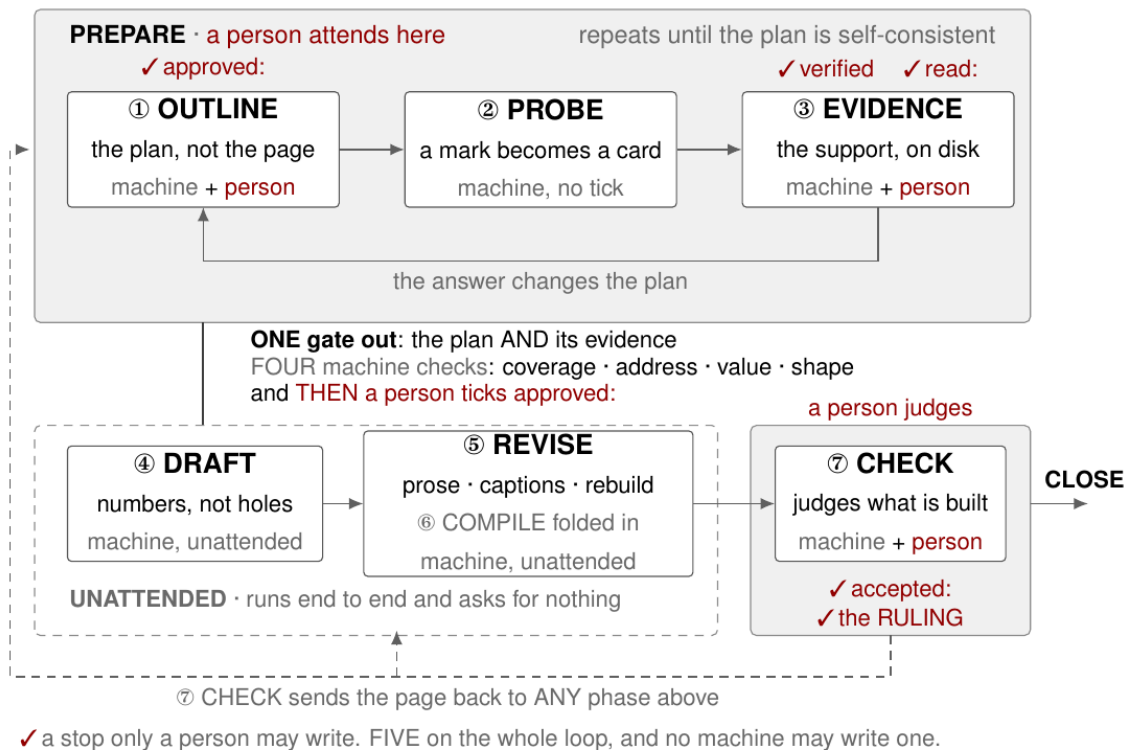


Figure 1. The loop at a glance, in the shape ruled 260819. A PREPARE loop (172 OUTLINE, 173 PROBE, 174 EVIDENCE) repeats until the plan is self-consistent, because evidence does not confirm a plan; it changes it. ONE gate leads out: four machine checks (coverage, address, value, shape), then a person's approved:. Everything after the gate runs unattended (175 DRAFT, 176 REVISE with 177 COMPILE folded in) until 178 CHECK, which a person judges and which reaches CLOSE or returns the page to any earlier phase. Red ticks are the five stops reserved for a person; no machine may write one.

Its intake is frozen out of the seven phase contracts and both legal-route tables; nobody has accepted it yet, so accepted: still reads ■ . Two reading paths. A person who only RUNS the

loop reads §1, §8 and §14. A person implementing one phase reads that phase's division, then its contract file. The loop today: counted rather than described. How much of it has ever

EXECUTED is a different card's question: PP02 owns the run counts, and §12.2 quotes them.

Who executes each phase: QPw00²-Display6 (Figure 2) names them.

2 · haipipe

Display: QPw00-Display6-agents-and-skills
 · kind=figure · label=fig:qpw00-agents-and-skills

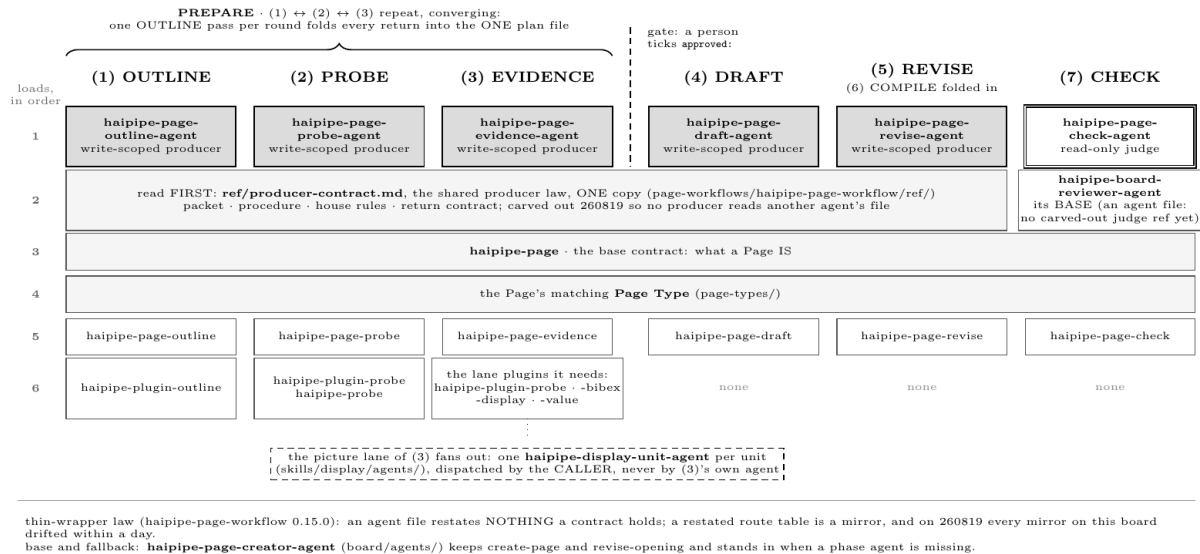


Figure 2. Who executes each phase, and what each loads: one agent per phase under page-workflows/agents/ (six files; (6) COMPILE is folded into (5), and (7)'s judge is haipipe-page-check-agent), each walking one fixed stack whose shared law lives once in ref/producer-contract.md. The PREPARE trio (1)(2)(3) repeats until the plan converges and stops at a person's approved: tick. The full routes and stop conditions are Display4's and Display1's.

Six agent files sit under page-workflows/agents/, five write-scoped producers plus ⑦'s read-only judge, and each walks one fixed skill stack whose shared law lives once in ref/producer-contract.md.

The vocabulary rule: the loop needs all four and none of them substitutes for another, which is why the word stays "phase" rather than "step". 📌 A phase is a TYPE and a step is an INSTANCE of one: in the one live run the single CHECK phase occupies steps 1, 3 and 5. All four in one receipt: none may be renamed onto another. (step: 4 · round: 1 · phase: CHECK · route: REVISE, four fields in one object) Renaming phase to step would put two different meanings on one key inside a single receipt, and the auditor reads both. step is the monotonic position that never recurs; phase is the kind of work that must be free to recur, which is the property the whole loop is built on. Why not "step": it would contradict the router's own name. (RUN is deliberately not ADVANCE, §12.1) The word must permit repetition, because CHECK may route back to any earlier phase, PROBE may be skipped entirely, and a page may re-enter DRAFT in a new round. The one live run proves it rather than asserting it: CHECK ran three

times and REVISE twice inside a single round, so a vocabulary that numbered them 1..7 could not describe what happened. The folder settles one of them: it already carries the workflow word. (board/page-workflows/, one skill per phase plus the head router) The directory says which of the four words scopes the family, and the contracts inside it say which authority each phase holds. So the full term is "workflow phase" where disambiguation is needed and "phase" everywhere else, and the 927 occurrences of the word across this board and the skill tree stay as they are. Two more words ride along, defined here and spent everywhere below: a PAGE persists through all of it, and an OPERATION is one edit act upon it: phases are named by reason, operations by shape. The four words are defined at the top, before any division spends them. What keeps the siblings honest is a sweep of the family's contracts for a phase called a step or a step called a phase, and A1.2 tracks it.

2 · OUTLINE ①: THE HEAD OF A LOOP THAT CONVERGES

The OUTLINE contract: what it decides, what it leaves alone, and what ends it.

The deliverable is the section list, the paragraph under each section, the bullets under each paragraph, and what each bullet still owes. It all lives in ONE file, outline/<stem>-outline-v<N>.md, and the page's own prose stays untouched until DRAFT. OUTLINE marks a hole and stops there: raising cards and dispatching questions is PROBE's job.

The Aims are settled in the plan file itself, and not at DRAFT. Shape and target are read once and approved by one tick. Every bullet that owes something names its Aim by id, so no id can dangle, and DRAFT transcribes those Aims onto the page rather than inventing them. The plan carries both the Aim's id and its target, so a renumber moves them together and no id can point at a target the plan does not hold. The same placement is what lets PROBE run ahead of DRAFT. A probe card's stake is an Aim, so PROBE can start no earlier than the phase that settles the Aims. The Aims exist the moment the plan does, so the card carries its stake from the plan and waits for nothing.

Evidence changes the plan, and never just confirms it: this page's own PREPARE loop shows both directions: a planned division scored 0 of 4 on §10's split tests and was folded away, and a planned count of 17 was recomputed as 13 before the tick (both recorded in outline v3). Four machine checks run on every version of the plan. Coverage: every mark is served both ways. Address: every serves: resolves. Value: every quoted number recomputes. Shape: the divisions match the page type. The person is asked LAST, and for DIRECTION rather than arithmetic: the machine already proved the numbers, so the tick's question is whether the plan aims at the right thing. A tick belongs to the version it ticked. Quietly editing an approved plan is how serves: addresses go stale; a changed plan gets a new version and a new tick. What the gate sees on this board today: PP06.v1 counts 17 plans, and PP06.v2 finds 6 that pass all four checks. Coverage is the common failure (PP06.v3, 10 plans); address and shape fail none (PP06.v4 and PP06.v5, both 0); value fails one (PP06.v6). The plan is ONE file, so its folds serialize: PROBE and EVIDENCE fan out safely across separate folders, and one OUTLINE pass per round is the merge point. The FIRST check is the human's own look, ruled after this page's own live round: the look is cheap and early, and it catches structure and style before any card is raised or run is spent. The look is not the gate. approved: ends the loop, and it is taken only after the four checks pass.

No machine may write the tick: it is the loop's one hard human gate. A phase whose entire output fits on one screen, and which a person can reject in ten seconds, belongs in front of every expensive phase rather than folded into one. The file's shape, its marks and its version rules live at QPw1-outline and haipipe-plugin-outline.

3 · PROBE ②: TURNING A MARK INTO A CARD, AND ASKING

The PROBE contract: an approved mark leaves the plan, becomes a card, and is asked. 

PROBE turns an approved mark into a card and asks; landing what comes back is ③

EVIDENCE's job.

The card holds the question, its serves: backlink into the plan, and later its ## Values; the question goes out stripped of stake. PROBE is the only phase that may create probe/PP<NN>-<slug>/: OUTLINE marks, and EVIDENCE fills. It looks before asking, MATCH before RAISE: an existing card on this page, a PageX match, or a QA-bank answer to the same question is reused, never duplicated. The trigger, the four-step MATCH lookup, and the optional-when-no-unknown-is-hidden rule live at QPw3-probe.

Filing a QA-probe does not name the target Page: the probe layer never learns which page consumes the answer, and that wall is what keeps executors stake-blind. A family may file the QA-probe under an evidence route such as Literature or Value without transferring the Q-consumer, the A-consumer, or the State to that route's topic Page. Treating physical placement as an ownership transfer gives one exchange two Page-facing consumer surfaces before the answer reaches prose. One active Page bounds the consumer write: a PROBE run writes consumer rows only into the one page it was opened for. When the same Q-executor serves other Q-consumers, the Probe surface records their references and makes the answer available, and each sibling page interprets it through its own PROBE or REVISE route, with its own version and CHECK. This preserves one Q-executor for reuse without turning one answer into an unbounded cross-Page edit. One reference shows the whole chain: from the bullet's mark to the card, to the QA file, to the pulled proof, every hop is a recorded pointer, so a read-only projection can render the chain without a second authored copy. Missing targets, superseded answers, or changed source hashes must fail visibly rather than leave an old projection looking current. One edge stays open, and it is recorded rather than hidden: whether a family adapter may transfer Q-consumer ownership after handoff is the open Decision Now row below, and A3.2 waits on it.

4 · EVIDENCE ③: LANDING WHAT CAME BACK

The EVIDENCE contract: what enters, the lanes it runs, and what ends it.

QPw00³-Display5 (Figure 3) draws the division at once, and its head is the probe card: the plan ASKS, the card answers, the

3 · haipipe
Display: QPw00-Display5-evidence-logic ·
kind=figure · label=fig:qpw00-evidence-
logic

value lane fills the card itself, a data display freezes FROM the card's proof/, and the citation lane is a person's and does not pass through the card.

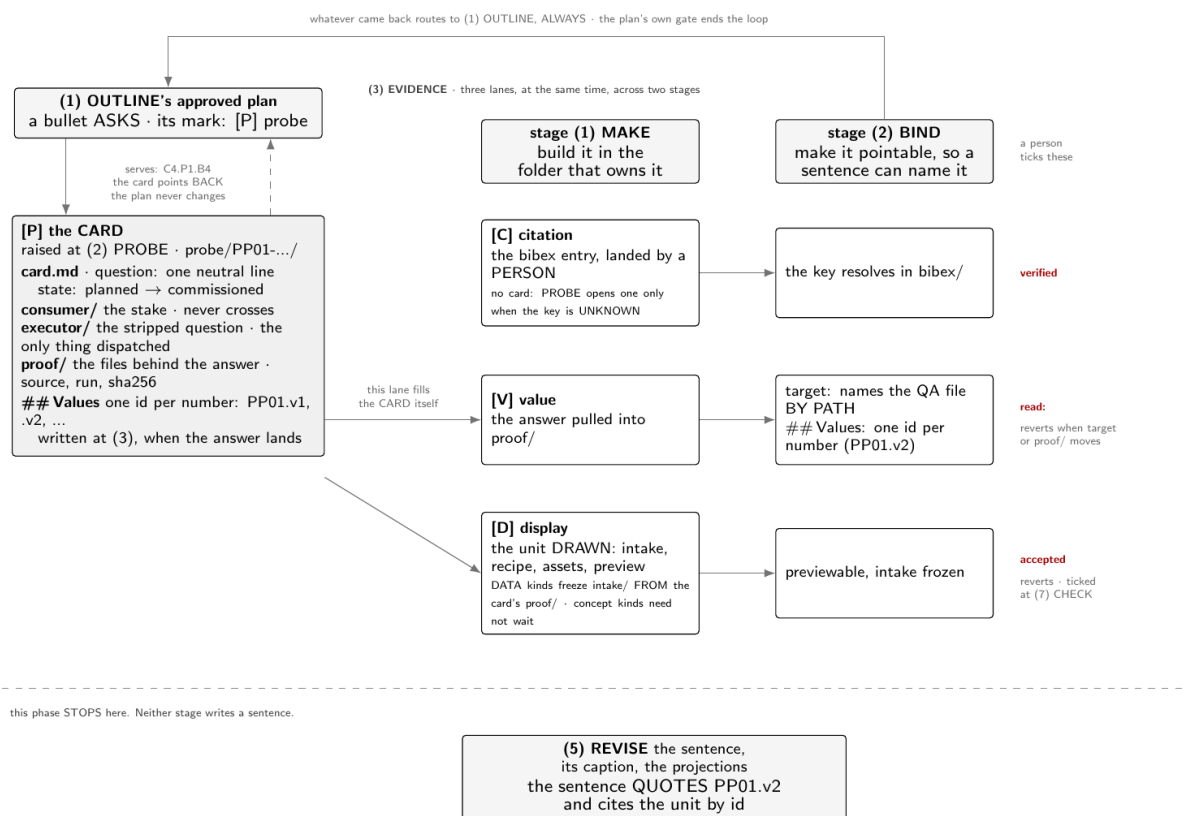


Figure 3. EVIDENCE, whole, with its head: the plan's bullet ASKS, PROBE raises the card (consumer/, executor/, proof/, and one id per value), and the three lanes run through MAKE and BIND at the same time. The value lane fills the card itself; a DATA display freezes its intake from the card's proof/; the citation lane is a person's and does not pass through the card. Two ticks revert; accepted is CHECK's. Neither stage writes a sentence; REVISE does, quoting one value by its id.

Neither stage touches this page's ## Content. That is the whole boundary: EVIDENCE changes what the page KNOWS and REVISE changes what it SAYS. A phase that both landed the answer and wrote the sentence could not be audited, because the only evidence that the answer came first would be its own report.

A 📌 mark sits on a bullet that ASKS. The bullet keeps only the question; the answer arrives later as an append plus the card's values. One card may serve many bullets, and its serves: line lists every address it answers. The ask and the quote share one probe/ folder: 📌 marks the ask,

marks a quote of one landed number, and both point into the same card. The live example is one division up: §1.2's counts stand on PP01, whose card carries the question, its state, and the three landed values.

What it lands: a number bound BY PATH to the QA file that answered it, with the pulled file sitting in the card's own proof/. One card, many values. A card is ONE question, and its answer usually holds several numbers, so the id goes one level deeper: allocated in card.md's ## Values block when the answer lands, and never before. That is what makes two failures visible instead of silent: a number in the prose citing no PP<NN>.v<n>, and a value in a card that no sentence uses. Both directions are checked, and the  tab reports them: a surface with no storage, because the number already lives in proof/ with its source, run and sha256. The live example: this page states 7 declared phases from PP01.v1, 6 contracts from PP01.v2, and 5 person-reserved ticks from PP01.v3. The card's v4 row reads NOT HERE and points at PP02, because run counts are that card's question, and its read: still reads , so every one of these quotes still waits on a person.

What it lands: one entry in bibex/<stem>.bib, and a key a sentence can cite. verified never reverts because provenance does not decay: it records that a person read the entry against its source. The live example is this page's own store: (Luo et al. 2026⁴) was transcribed verbatim from QPf4-chat's bibex and carries its verified = {<who> <date>} signature, so a sentence here may cite the key and its chip resolves green.

4 · haipipe

Citation: Junjie Luo, Xuzhe Zhi, Rui Han, Abhimanyu Kumbara, Mansur E Shomali, Anand K Iyer, Ritu Agarwal, and Guodong Gordon Gao. Eventglucose: A benchmark for continuous glucose monitoring forecasting with behavioral event context. In Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2, pages 9510–9521, 2026. key=luo2026eventglucose

What it lands: a DRAWN, previewable unit, not a folder of material. Steps ② to ④ live in this lane because a display's return must be as directly usable as the other lanes': a citation lands a key, a value lands a bound number, so a display lands a drawn unit rather than an unrendered folder. Which units wait depends on their kind: a DATA unit (table, figure) freezes its intake FROM a probe card's proof/ and waits for an answer, while a CONCEPT unit (diagram,

5 · haipipe

Display: QPw00-Display3-who-does-what · kind=figure · label=fig:qpw00-who-does-what

tex, illustration) freezes a listing of source files and waits for nothing. The intake manifest pins every source with a sha256, so a moved or edited input is computable staleness, and accepted:

✓ reverts with it: acceptance binds to the frozen inputs it was accepted with. The live example is QPw00⁵-Display3 (Figure 4), one glance down at §14: a concept unit with 15 frozen inputs in its intake/manifest.yaml, accepted: still , and cited both here and by §14's table, which is legal because a unit owes AT LEAST one citing sentence, not exactly one.

#	PHASE	WHAT HAPPENS	YOUR JOB	TIME	HOW OFTEN
<i>once per page</i>					
0	OPEN	haipipe-page CREATE copies the page template and registers the page in board.md (a VERB, not a phase)	YOU write the title so it states the page's PURPOSE	15 min	once
<i>the loop — CHECK may send you back to rows 1, 2, 3, 4 or 5</i>					
1	OUTLINE	a machine writes the SHAPE only, into outline/...-v<N>.md and nothing in the page itself; then FOUR checks run: coverage, address, value, shape	YOU judge the plan's DIRECTION, never its arithmetic, then tick approved: ← <i>the four run</i> <i>BEFORE you are</i> <i>asked; this is the</i> <i>loop's one gate</i>	10 min	○ each pass
2	PROBE	MATCH first (this page, then PageX, then the QA bank), and only then a card at probe/PP<NN>-<slug>; the stripped Q-executor is the only thing that crosses	nothing	0	○ each pass
3c	CITATION	a machine may SUBSET or TRANSCRIBE a real record and may never COMPOSE one	YOU land the entry verbatim in bibex/ , so it carries verified = {WHO YYMMDD}	20 min	○ per entry
3v	VALUE	the bank answers into its own QA file, target: names that file BY PATH, and checks/values.py re-runs the number	YOU tick read: , which now means "I agree with the judgment inside the question" and not "I checked the arithmetic" ← <i>it reverts when</i> <i>target or proof moves</i>	30 min	○ per card
3d	DISPLAY	this lane DRAWS since 260819: intake, recipe, the pick, then assets/ and preview.pdf . It returns a previewable unit rather than a folder	YOU rule intake/ and choose among candidates/ ← <i>no tick here:</i> <i>accepted: is row</i> <i>7a's</i>	?	○ per unit
Rows 1, 2 and 3 REPEAT AS ONE UNIT until the plan and its evidence agree, and 3c, 3v and 3d run at the SAME TIME. Three of the five person-reserved ticks are in this band; the other two are 7a and 7b.					
4	DRAFT	it now ENTERS on evidence that already landed, so it writes the NUMBER; a hole is the EXCEPTION and	nothing	0	per round

	the PROBE PROBE and must name the input that is missing		
5 REWIRE	the sentence that cites each drawn unit by id, the caption for THIS page's claim, and the latex/ and word/ rebuild. It no longer draws	nothing	0 $\odot$ each round
6 COMPILE	folded into 5 since 260819 and holding no contract of its own; whether the fold is permanent is QPw5-revise's open ruling	nothing	0 with 5
<i>4, 5 and 6 run END TO END and stop for nobody: a run that pauses for a person between DRAFT and CHECK is stopping in the wrong place.</i>			
7 CHECK	a DIFFERENT actor judges the RENDERED page and the BUILT artifact, and may not repair its own finding in the same pass	YOU take CLOSE, or name where to go back $\hookrightarrow$ REWIRE, DRAFT, EVIDENCE, PROBE or OUTLINE	15 min $\odot$ each round
<i>what ends it — both ticks below are row 7 CHECK's own</i>			
7a ACCEPT	you are shown this gate only once the page's mechanical errors are ZERO, so it is a confirmation and not an inspection	YOU tick accepted: in each unit's README $\leftarrow$ a changed <i>intake/ drops it back</i> to $\square$	10 min per unit
7b CLOSE	CLOSE is a ROUTE and not a phase, and only verdict: pass may take it	YOU sign the Page Type's declared RULING $\leftarrow$ <i>silence is not</i> <i>consent</i>	5 min per round

TIME is an ESTIMATE and nothing else in this table is. The receipt shape carries no start/end pair, so no receipt can time a phase and not one phase here has ever been timed. Across this board's four stored runs the receipts cover OUTLINE, PROBE, DRAFT, REWIRE and CHECK (PROBE's first two landed only on 260819), and EVIDENCE and COMPILE had none when this table was frozen that evening. Row 3d carries no estimate at all, because it took over three of REWIRE's steps on 260819 and nobody has priced the lane since. Every other cell is transcribed from a file frozen in intake/inputs/.

Figure 4. The page workflow as a person experiences it, in the order it has run since 260819: the PREPARE loop (1 OUTLINE, 2 PROBE, 3 EVIDENCE) repeats as one unit before 4 DRAFT, and 5 REWIRE with 6 COMPILE folded in, ever start. EIGHT of the twelve rows ask something of a person and four read nothing; of those eight, five carry a tick a machine may never write, and three (0 OPEN, 3d DISPLAY and 7 CHECK) ask for a judgment that leaves no tick behind. The four nothing rows are 2 PROBE and the whole unattended stretch 4, 5, 6. TIME is the only estimated column, because no phase on this board has ever been timed.

They run at the same time, and none waits for another: the fold at ① OUTLINE is the only merge point. Each return is directly usable by a sentence: a citable key, a quotable id, a previewable figure. Three ticks live here and two of them revert: verified never, read: when target: changes or proof/ is re-pulled, accepted: when the frozen intake changes. The ticks belong to stage ② BIND, because a tick is what turns a made thing into one a sentence may quote; accepted:  is the exception and stays at CHECK, since what it judges is the drawn artifact as a reader meets it. EVIDENCE on this board, counted: PP03.v1 finds 23 probe cards, PP03.v2 13 still at planned, PP03.v3 0 read by a person, PP03.v4 10 of 10 display units rendered, and

PP03.v5 0 accepted. Every person-reserved tick in this phase is open across the whole board. The lanes are real folders under real contracts with almost no traffic, and saying so is the difference between a contract and a description. The lane contracts are QPw4c-citation, QPw4v-value and QPw4d-display, under the phase contract QPw4-evidence.

5 · DRAFT ④: TURNING EACH POINT INTO SENTENCES

The DRAFT contract: what enters, the one conversion it performs, and what it hands forward. DRAFT enters on landed evidence, so it writes the NUMBER itself rather than a placeholder: the PREPARE loop already landed everything a sentence needs. A hole is the exception, not the normal case, and it names the input it is missing rather than just marking a gap. The Aims are settled at OUTLINE, and this phase transcribes them onto the page rather than inventing them. It executes the approved plan and names nothing new: a shape found wrong during DRAFT goes back to OUTLINE as a new version, never a quiet local fix. And it is named by reason, never by diff: the same edit shape can be DRAFT or REVISE, and only the reason distinguishes them.

QPw00⁶-Display7 (Figure 5) draws the conversion in both directions: one approved POINT becomes one or more sentence scaffolds, and the join survives later edits because it is carried twice.

6 · haipipe

Display: QPw00-Display7-point-to-sentence · kind=figure · label=fig:qpw00-point-to-sentence

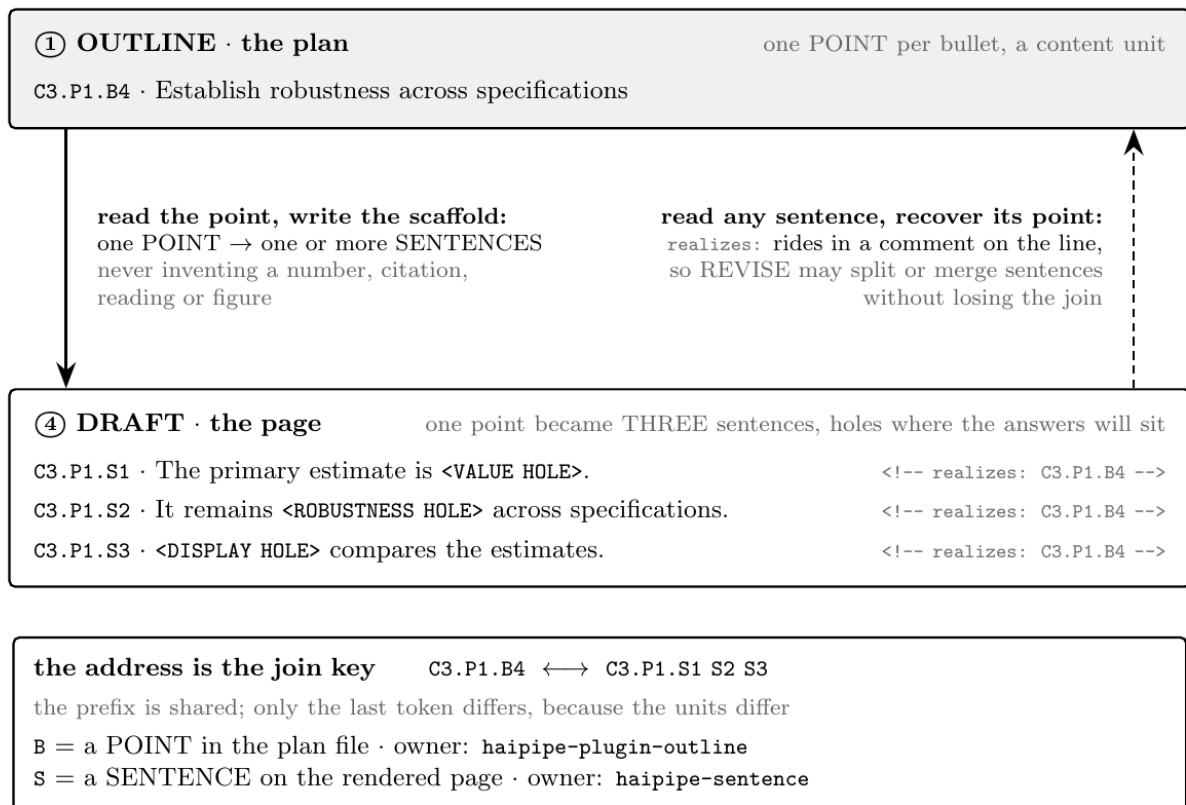


Figure 5. The point-to-sentence conversion at 4 DRAFT, shown both ways round. Forward: one approved outline POINT (C3.P1.B4) becomes one or more sentence scaffolds (C3.P1.S1–S3), each hole sitting exactly where its answer will sit, and DRAFT may never invent the value, citation, reading or figure. Backward: the machine-readable realizes: comment on each scaffold line recovers the point from any sentence, so REVISE can split or merge sentences without losing the join. The shared C3.P1 prefix is the join key; B (a point in the plan) is owned by haipipe-plugin-outline, S (a sentence on the page) by haipipe-sentence, and neither redefines the other.

One bullet is a POINT, not a sentence, so the two sides count different units and say so. C3.P1 is shared, and only the last token differs: B for a point in the plan, S for a sentence on the page. The backlink also rides in a comment on the scaffold line, <!-- realizes: C3.P1.B4 -->, so REVISE may split or merge the sentence later without losing it. When a question stayed genuinely BLOCKED, the sentence keeps a visible hole carrying the card that owes it, so a reader sees what is missing and who owes it; §14.1's PP05 hole is this page's one worked example. A sentence may never invent a value, a citation, a reading, or a rendered figure: a sentence that already knows its answer was written after the fact, and holes keep the order honest.

One idea per sentence, and a sentence past about thirty words is usually two. The haipipe-writing plain rules apply to scaffolds, not only to finished prose. The AI-tell catalogue at [haipipe-writing/ref/ai-tells.md](#) applies HERE and not only at REVISE, because machine cadence written at DRAFT gets rewritten twice instead of once. What a scaffold may not contain, as a list a writer can check against: an invented number; a hedge standing in for a hole; a summary of what a card says, which the 🧭 tab already reads live; or a division the plan did not name. Each forbidden thing is a way of smuggling unlanded evidence into prose. The full contract and its worked example live at [QPw2-draft](#).

6 · REVISE ⑤: TURNING LANDED EVIDENCE INTO SENTENCES

The REVISE contract: the Page changes while its purpose and Aims remain fixed. 📌 REVISE improves the realization of the current round rather than redefining its promise.

REVISE improves the realization while purpose and Aims stay fixed; if either moves, it is not REVISE, and the work reopens OUTLINE in a new version or DRAFT in a new round. It is the phase that fills a landed hole: the hole is replaced by the number plus its id, never by a restatement of the card. Its fixed-promise test, its five-step LAND, ARGUE, SOUND, CITE, BUILD order, and the latex/ and word/ rebuilds live at [QPw5-revise](#), which also carries COMPILE. This page's own §9 still carries the shared DRAFT-versus-REVISE test table that separates the two phases on the same edit, so a reader is pointed sideways as well as forward. Scored against §10's four split tests, rather than asserted: COMPILE rebuilds latex/ and word/ from the source as it now stands. That is real work with a real audit code, projection-stale, and it is the only member of the loop with no contract file, no receipt in any stored run, and no tick. The loop declares seven phases and the Diagram draws ⑥, because that count is cited across the contracts. COMPILE gets no division at the same level as the six phases that have one, because a heading of its own would assert a symmetry the four tests refuse. Whether the fold is permanent is [QPw5-revise's](#) open ruling.

7 · CHECK ⑦: WHERE THE CURRENT VERSION GOES NEXT

The CHECK contract: one concrete version is judged against its promise and routed.  CHECK observes and decides; another phase performs the content change it requests.

CHECK judges one exact version and routes it: CLOSE, or back to the phase that owns the fix. Only CHECK closes; producers may HOLD or hand forward, and the judge alone ends a page's run. It judges and never repairs, because a fix by the finder would erase the independent finding; repair returns to a producer. The judge-may-not-repair separation, the three independent built-artifact counts it reads, and the accept-biased human gate live at QPw6-check: unclear findings route forward for repair rather than blocking the page. This page's own §13 still carries the receipt fields and invariants CHECK's routed findings are measured against, so nothing here is left an orphaned reference.

8 · TRANSITIONS FORM ROUNDS, NOT A RIGID CONVEYOR BELT

The transition grammar: repetition is legal, and returning to DRAFT changes the round.  The arrows express dependencies and routing choices, not a rule that every phase runs once.

The common order is not the only legal order: rounds may repeat phases or skip them, and the route table defines legality, not habit. DRAFT, DRAFT, PROBE, REVISE, DRAFT is a legal history: two repeated design passes in one round, an optional question, a repair, and a final DRAFT that means revision reopened the promise and began a new round. REVISE to DRAFT is that restart, not a forbidden edge: the Page persists, earlier evidence remains available, and its relevance is re-checked against the new promise, while any earlier closing decision stops applying. A phase label requires a reason: "we edited prose" names no phase, "the aim moved" does, and without the reason two identical diffs can be mislabeled as different phases. These rules are what made the six-page split safe: six phase pages split out of this one, and the comparison and transition grammar they must agree on stays here, in §8 and §9, where one home keeps them auditable. The shape to take away is three bands, not eleven steps: OPEN happens

once, the PREPARE and writing phases repeat, and the CLOSE acts happen when CHECK stops the loop; §14 prices the same shape as work.

9 · OPERATIONS ROUTE BY REASON, NOT BY EDIT SHAPE

The same operation under two authorities: a compact test for ordinary Page changes.  The operation says what changed in the file; the authority says which phase performed it.

Adding and deleting name no phase: adding a paragraph to explain evidence for an existing Aim is REVISE, and adding one because the Page now answers a new question is DRAFT. Deleting unsupported or duplicate prose while keeping the Aim is REVISE; deleting the promised result itself, or its Aim, is DRAFT. Moving and rewriting use the same test: improving flow under the same argument is REVISE, and changing the argument the Page promises is DRAFT even when the diff is one moved heading. PROBE and CHECK produce records on different write surfaces, cards and verdicts, and neither touches page prose; when either causes target prose to change, the edit runs under REVISE or a restarted DRAFT. Sibling pages cite this page by § address rather than copying it (QPw1-outline pins QPw00 §6 and QPw6-check pins QPw00 §10 in their Files today), so a silent move of any division here, this table included, breaks their pins.

10 · ONE LIFECYCLE PAGE HOLDS THE PHASES TOGETHER

The Page split rule: each phase begins as a Content division and earns a Page only by becoming an independent question.  Shared boundaries stay together; a phase moves out only when it can carry and close a question of its own.

A phase begins as a Content division here and earns a Page of its own only by becoming an independently closable question: one it can close with its own evidence, its own gate, and its own continuation files. The four tests above decide it, and they are not left abstract: the re-cut that made this board's phase pages scored the phases against them, overturned this division's stay-together default, and split six pages out, QPw1-outline through QPw6-check, while COMPILE scored 0 of 4 and stays a step of §6; the Log keeps that ruling. Separate skills still do not require

separate design pages: a phase skill is executable law a worker loads one contract at a time, a design page is an argument, and one phase may have both or either.

11 · PAGE TYPE AND PAGE PHASE ARE SEPARATE SKILL AXES

The skill composition: the base resolves what the Page is and how the current work is acting on it. 📌 Page Type and Page Phase are orthogonal, so their folder names and skill names must not collapse them into one label.

for-* names only Page Types: the preposition says which persistent Page shape varies from the base, and the roster has grown to sixteen variants across six skill sets, with QB6 owning the admission test and the list. Phase skills use direct names, haipipe-page-<phase>, because a phase is an active authority and not a Page variant, and it applies across Page Types. PROBE keeps one vocabulary across the boundary: the target Page owns the stake-bearing Q-consumer, the phase strips it into a neutral Q-executor, binds the returned A-executor, and writes an A-consumer interpretation for each consumer; one Q-executor may serve several Q-consumers, and haipipe-probe remains the shared crossing protocol. The paths are one rename behind in places: contracts still name page-phases/, the retired folder name, and each is one mechanical sweep away from correct. The base-adoption question itself is closed as P1: option B won, the smallest reversible adoption, and the concrete router later earned the verb RUN rather than ADVANCE.

12 · RUN TURNS THE PHASE GRAMMAR INTO A BOUNDED LOOP

The executable flow: one controller composes separate producer, builder, and judge roles without prescribing one phase sequence. 📌 RUN follows returned authority routes and stops at explicit terminals or limits.

RUN means execute the current Page lifecycle from a named starting authority until CLOSE or HOLD; it is not ADVANCE, which would promise forward-only steps. The controller reads each phase's requested route and follows the legal-route table, which is dynamic: producers may repeat or hand off, and only CHECK may CLOSE. The packet bounds what agents may know: it

names the Page, its Type, the starting Phase, the run intent, source paths, settled constraints, the closing gate, and step and round limits; context beyond it is contamination. A missing source, unknown gate, or ambiguous authority becomes HOLD rather than an invented input. The controller itself only routes, because writing or judging from the controller would collapse the producer, builder and judge separation it exists to keep: the producer performs exactly one phase, the mechanical builder rebuilds and snapshots the version, and the fresh judge returns the verdict, testing for CHECK rather than listing producer phases. Every stop is honest: CLOSE, explicit HOLD, missing input, failed build, version mismatch, required human ruling, maximum steps, or maximum rounds; reaching a limit says the process did not converge within its budget, and it never says the Page passed.

QPw00⁷-Display1 (Figure 6) states the controller as an algorithm, so a reader can see the three actors alternate and read the stop conditions in the order the loop tests them.

7 · haipipe

Display: QPw00-Display1-run-loop ·
kind=figure · label=fig:qpw1-run-loop

```

packet :  $\Pi = (\text{page}, \text{type}, \varphi_0, \text{intent}, \text{sources}, \text{constraints}, \text{gate}, \text{limits})$ 
receipts:  $\rho = (r_1, \dots, r_n)$ , one receipt per attempted phase
1  $\varphi \leftarrow \varphi_0$  // an existing Page enters at CHECK, a new one at
   OUTLINE
2  $s \leftarrow 0$ ;  $k \leftarrow 1$  // steps taken, and the current round
3 while  $s < \text{limits.max\_steps}$  and  $k \leq \text{limits.max\_rounds}$  do
4    $\Gamma \leftarrow \text{CONTEXT}(\text{page}, \varphi)$  // one hop, this phase's rows only;
   a bad row is a HOLD
5   if  $\varphi \neq \text{CHECK}$  then
6      $(v', \hat{r}) \leftarrow \text{Producer}(\varphi, \Pi, \Gamma)$  // one phase, by one actor,
       who also proposes a route
7      $v' \leftarrow \text{Builder}(v')$  // rebuild, deterministic checks,
       version identity
8   else
9      $(v', \hat{r}) \leftarrow \text{Judge}(v, \Pi, \Gamma)$  // a fresh read-only actor,
       judging one exact version
10   $r \leftarrow \text{RECEIPT}(\varphi, \text{actor}, v \rightarrow v', \hat{r})$ ; append  $r$  to  $\rho$ 
11  if  $\hat{r}.\text{route} \notin R(\varphi)$  then return  $\rho$  with HOLD
12  if  $\hat{r}.\text{route} \in \{\text{CLOSE}, \text{HOLD}\}$  then return  $\rho$ 
13  if  $\hat{r}.\text{route} = \text{DRAFT}$  and  $\varphi \notin \{\text{OUTLINE}, \text{DRAFT}\}$  and  $\hat{r}.\text{reopens}$ 
     then  $k \leftarrow k + 1$ 
14   $\varphi \leftarrow \hat{r}.\text{route}$ ;  $v \leftarrow v'$ ;  $s \leftarrow s + 1$ 
15 return  $\rho$  with LIMITREACHED // did not converge, which never
    means the Page passed

```

Figure 6. RUN, as the router it is rather than the conveyor belt its name could suggest. The producer, the builder, and the judge are three separate actors; the loop follows the route each returns, refuses a route outside $R()$, and stops at CLOSE, HOLD, or a limit.

QPw00⁸-Display2 (Figure 7) writes the legal-route table as six explicit row-sets and derives the laws this prose would

8 · haipipe
Display: QPw00-Display2-route-relation ·
kind=figure · label=fig:qpw1-route-relation

otherwise assert: PROBE and EVIDENCE route only sideways or BACK to OUTLINE, so OUTLINE's gate is the one door into DRAFT; and CHECK cannot route to CHECK, so a judged version reaches a second judgment only through a producer.

$\Phi = \{\text{OUTLINE, PROBE, EVIDENCE, DRAFT, REVISE, CHECK}\}$ the six, in run order
 $T = \{\text{CLOSE, HOLD}\}$ the two terminals

COMPILE is not a member. Since 260819 it has no contract of its own, it is folded into REVISE, and no phase routes to it.

$R(\varphi)$ row by row, and the point is that it is **not** uniform: the first three rows are a LOOP that can only leave through one door.

PREPARE – repeats until the plan is self-consistent

$R(\text{OUTLINE}) = \{\text{OUTLINE, PROBE, DRAFT, HOLD}\}$

$R(\text{PROBE}) = \{\text{PROBE, EVIDENCE, OUTLINE, HOLD}\}$

$R(\text{EVIDENCE}) = \{\text{EVIDENCE, OUTLINE, HOLD}\}$

EXECUTE – entered only through OUTLINE's gate

$R(\text{DRAFT}) = \{\text{DRAFT, PROBE, REVISE, CHECK, HOLD}\}$

$R(\text{REVISE}) = \{\text{REVISE, EVIDENCE, DRAFT, CHECK, HOLD}\}$

$R(\text{CHECK}) = \{\text{CLOSE, OUTLINE, PROBE, EVIDENCE, DRAFT, REVISE, HOLD}\}$

Four laws follow from the table, and they are why it is worth printing:

$\text{CLOSE} \in R(\varphi) \iff \varphi = \text{CHECK}$	only a judge may close
$\varphi \in R(\varphi) \iff \varphi \neq \text{CHECK}$	every phase may repeat, not the judge
$\text{HOLD} \in R(\varphi) \text{ for every } \varphi \in \Phi$	every phase may stop honestly
$\text{DRAFT} \notin R(\text{PROBE}) \cup R(\text{EVIDENCE})$	the loop's one door out

The two code files AGREE with this table today. The controller ([ref/page-lifecycle.workflow.js](#), **LEGAL**) and the auditor ([src/page_lifecycle.py](#), **LEGAL.ROUTES**) encode these same six rows: all three PREPARE edges are legal, and neither PROBE nor EVIDENCE routes forward. One residue, documented in both files' own comments: each keeps a COMPILE row and a REVISE→COMPILE edge so an already-stored receipt naming COMPILE stays auditable. No phase contract names those edges.

Figure 7. The legal-route relation R after the 260819 loop change, six rows, and the four laws the table makes checkable. The rows are deliberately NOT uniform: PROBE and EVIDENCE route only sideways or BACK to OUTLINE, so the single door out of the PREPARE loop is OUTLINE's own gate, four machine checks plus a person's approved:. The judge alone reaches CLOSE and alone cannot reach itself, so every judged version passes through a producer before it is judged again; every phase without exception may stop at HOLD. The controller and the auditor encode this same table since 260819, keeping only a Compile row no contract names, so already-stored receipts that name it stay auditable.

The runs so far, counted: PP02.v1 finds 4 runs stored under `_runs/page/`, PP02.v2 24 receipts across them, PP02.v3 6 of 7 phases covered, and PP02.v4 exactly one phase never executed, COMPILE. So the loop is fully specified and all but one phase exercised, and the one phase with no receipt anywhere is also the one with no contract of its own.

Each phase is executed by its own producer: the controller's PRODUCER_AGENTS table maps a phase to haipipe-page-<phase>-agent, with the creator agent as the fallback. The agent files stay thin wrappers: each carries identity, skill chain, walls and receipt duty, and the shared law lives ONCE in ref/producer-contract.md, because an agent file that restates a contract is a mirror, and mirrors drift. Counted on disk: PP07.v1 finds 6 agent files under page-workflows/agents/, PP07.v2 5 phases owning a producer of their own, PP07.v3 2 sharing or borrowing one (COMPILE and CHECK), and PP07.v4 4 support agents beside them. The roster itself, with each unit's daily debts, lives on QPw00a: this page owns the RULE, because a mirror of the roster here would drift within a day.

13 · AUDIT PROVES PROCESS CLAIMS WITH RECEIPTS AND FAULT TESTS

The assurance model: deterministic invariants, fresh semantic judgment, and direct or human evidence support different quality claims. 📌 A passing process audit proves that the declared process ran correctly, not that every possible substantive claim is true.

Each receipt binds one attempted phase to its actor, its builder, its status, the source and render SHA-256 identities before and after, the route it requests, and its reason, artifacts, evidence, findings and human-gate state; CHECK additionally binds checked_version and a verdict. Receipts are ordered and stored outside Page discovery under _runs/page/<page-id>/<run-id>.json, and the deterministic auditor independently rehashes the artifacts on disk, so a claim repeated inside a receipt is never accepted as its own proof. Seven invariants make the loop auditable, among them version continuity from receipt to receipt, one writer per pen with distinct actor identities per role, legal routes only with CLOSE reserved to CHECK, packet and limit matching, and a human gate that closes only on durable evidence. Testing covers both directions: the happy paths and branch routes must pass, and injected faults, self-approval, mutation after CHECK, illegal routes, broken continuity, symbolic hashes, missing human evidence, must each be rejected for the specific invariant they violate. Quality is evidenced, never declared: a green audit proves the declared process ran, not that every substantive claim is true, and silence is

never a pass. The auditor, run on this board's own receipts rather than described: PP04.v1 audits 4 stored runs, PP04.v2 finds 0 that PASS, PP04.v3 8 findings, PP04.v4 5 distinct fault codes, and PP04.v5 exactly 1 real contract violation. The real one is checked-version-mismatch, on a stored CHECK receipt: CHECK must leave version_before, version_after and checked_version identical, because a judge reporting on a version it did not read is the one failure the receipt exists to catch. No reasoning predicted it; running the auditor found it. Most of the rest is expected noise, artifact-version-mismatch on pages edited after their runs closed, which is the recorded cost of auditing live pages rather than frozen ones.

14 · THE SAME LOOP, PRICED: WHAT A PERSON IS ACTUALLY ASKED TO DO

The cost rule: a reader deciding whether to run this loop needs the WORK, not the authority, so the loop is stated once more as one row per step, with a person's job and its price in it. ⚠ The row numbers here are the WORK's, not the loop's. Row 0 OPEN is a verb, and rows 7a ACCEPT and 7b CLOSE are ⑦ CHECK's own ticks; rows 3c, 3v and 3d are the three lanes inside phase ③. Rows 1, 2, 4, 5 and 7 are phases ①, ②, ④, ⑤ and ⑦, and phase ⑥ COMPILE holds row 6 only to say out loud that it is folded into row 5. 📌 EIGHT of the twelve rows ask something of a person and four read nothing (2, 4, 5, 6); of those eight, five carry a tick a machine may never write (1, 3c, 3v, 7a, 7b), and three (row 0 OPEN, row 3d DISPLAY, row 7 CHECK) ask for a judgment that leaves no tick behind. Every asking row is the OPEN that starts the page, a step of the PREPARE loop, or CHECK: the loop's whole human cost is paid BEFORE DRAFT and AFTER REVISE, and never inside them. That is the same fact §12 states as authority, priced instead. QPw00-Display3 renders this table for print, with the fifteen files it is transcribed from frozen beside it.

The one measured number is zero: PP05.v1 reports 0 phases with a measured duration, and PP05.v2 names the 2 inputs that block it. Every phase's real duration is <HOLE: PP05 blocked, no stored receipt carries a start/end pair, and COMPILE has no receipt at all (PP02.v4)>, so the TIME column stays an estimate on its own face. Block ① is the one to fix first, because it is a

contract change rather than work: the receipt shape in `ref/page-run-contract.md` needs a per-phase start and end stamp. Block ② then clears by running the loop once end to end. An estimate was not substituted, because substituting one is how this column got its current values. Every other cell is transcribed from a file frozen in `display/QPw00-Display3-who-does-what/intake/inputs/`: the manifest names all fifteen with the sha256 of the copy AND of the live source, so `checks/intake.py` can recompute staleness on demand. Row 3d carries no estimate at all, because nobody has priced the lane's render, pick and build steps. A number nobody has measured is marked as an estimate on the table's own face rather than left to look like the others. Once receipts carry start and end pairs, the column is rebuilt from `_runs/page/` timestamps instead of estimated, which is A14.2's whole test.

REFERENCES

Junjie Luo, Xuzhe Zhi, Rui Han, Abhimanyu Kumbara, Mansur E Shomali, Anand K Iyer, Ritu Agarwal, and Guodong Gordon Gao. Eventglucose: A benchmark for continuous glucose monitoring forecasting with behavioral event context. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, pages 9510–9521, 2026.